

ALERTA SATIPO

Guía de instalación y configuración

Windows · PowerShell · Supabase | Versión 1.0 | 25 de septiembre de 2026

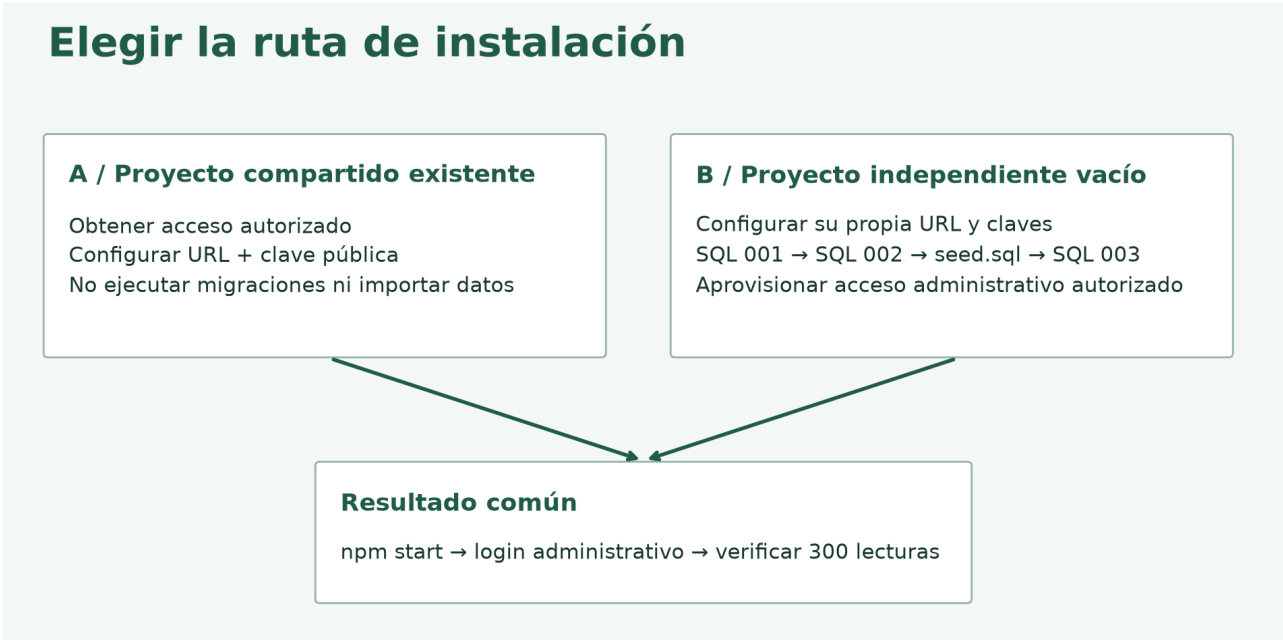


Figura 1. Rutas de instalación. Diagrama explicativo, no captura de ejecución.

Procedimiento para ejecutar el proyecto existente o preparar un entorno independiente. Incluye verificaciones, resolución de errores y publicación opcional.

Contenido

1. Alcance y elección del entorno
2. Requisitos previos
3. Descargar e instalar dependencias
4. Configurar la conexión local
5. Ruta A — Usar la base existente
6. Ruta B — Preparar un proyecto vacío
7. Iniciar la aplicación
8. Validar la instalación
9. Publicación opcional en Firebase
10. Solución de problemas
11. Criterios de finalización y referencias

1. Alcance y elección del entorno

Esta guía instala el panel administrativo existente: HTML, CSS y JavaScript, autenticación y PostgreSQL en Supabase. Firebase Hosting aloja la versión publicada. No se instala Express ni una arquitectura hexagonal: esa propuesta corresponde a documentación de diseño independiente.

Ruta A — Base existente: para integrantes autorizados que ejecutarán el proyecto localmente utilizando la base compartida. No ejecutar migraciones, importar registros ni crear cuentas.

Ruta B — Proyecto nuevo: para un entorno independiente y vacío de Supabase. Crear el esquema y cargar la muestra en el orden descrito. Los estados que se cambien en la ruta A afectan a la base compartida; utilizar B para pruebas aisladas.

El resultado esperado es un acceso exclusivo para administradores, 300 lecturas históricas, seis nodos virtuales y sus casos y tareas demostrativas. No representa sensores físicos conectados ni reparaciones reales.

2. Requisitos previos

- Git y acceso al repositorio privado de GitHub.
- Node.js 22 actualizado a una versión de mantenimiento reciente, con npm. El proyecto usa --env-file-if-exists; las primeras versiones de Node 22 pueden no reconocer esa opción.
- Python 3.10 o superior; el servidor usa Path.is_relative_to. La preparación de datos no requiere paquetes Python adicionales.
- Visual Studio Code u otro editor, navegador e Internet para consultar Supabase.
- Ruta A: URL y clave pública del proyecto, más una cuenta administrativa existente del aplicativo. Pertenecer al equipo de Supabase no otorga automáticamente acceso al panel.
- Ruta B: permisos para crear y administrar un proyecto Supabase y una cuenta de aplicación autorizada por el responsable del entorno.

Comprobar las herramientas en una terminal nueva:

```
git --version
node --version
npm --version
python --version
```

Resultado esperado: cada comando muestra su versión. Si PowerShell bloquea npm.ps1, utilizar npm.cmd en lugar de npm; no es necesario cambiar globalmente la política de ejecución. No se requiere Docker ni PostgreSQL local para la ruta documentada con Supabase alojado.

3. Descargar e instalar dependencias

En una carpeta de trabajo, ejecutar:

```
git clone https://github.com/76778920-tech/alerta-satipo.git
Set-Location alerta-satipo
npm ci
npm run build
```

La autenticación de Git debe utilizar una cuenta invitada al repositorio. No introducir tokens dentro de la URL del comando. Si el repositorio ya está descargado, abrir su carpeta y conservar los cambios locales antes de actualizarlo; no es necesario clonarlo de nuevo.

npm ci instala las versiones fijadas en package-lock.json. npm run build genera shared/vendor/supabase.js. Resultado esperado: mensaje «SDK de Supabase compilado localmente».

4. Configurar la conexión local

Comprobar si ya existe `.env` antes de copiar la plantilla:

```
Test-Path .env
```

Solo si devuelve False:

```
Copy-Item .env.example .env
```

Editar el archivo. Para la ruta A:

```
SATIPO_MODE=supabase
SUPABASE_URL=https://ddfmooylklgwnrmlxqzc.supabase.co
SUPABASE_PUBLISHABLE_KEY=<clave_publica_del_proyecto>
SUPABASE_SECRET_KEY=
```

Sustituir el marcador de clave pública por la clave publishable o anon del mismo proyecto. Para la ruta B, sustituir también la URL por la del proyecto nuevo. La URL del CSV de GitHub no va en `SUPABASE_URL`: es una fuente de importación, no la conexión de la aplicación.

La clave privada no es necesaria para ejecutar el panel. Solo se necesita para operaciones administrativas opcionales como importar mediante el script. Nunca colocar una clave secret o service_role en `SUPABASE_PUBLISHABLE_KEY`.

```
npm run configure
```

Resultado esperado: `config/public.json` generado y mensaje de modo supabase. No editar ese JSON manualmente: regenerarlo al cambiar `.env`. La plantilla viene en modo demo; es necesario cambiarlo a supabase para esta guía.

5. Ruta A — Usar la base existente

El esquema y los 300 registros ya están cargados. No ejecutar `seed.sql`, `bootstrap-admin.mjs` ni las migraciones. Solicitar al responsable una cuenta de aplicación previamente autorizada; no existe registro público.

Proyecto: <https://supabase.com/dashboard/project/ddfmooyklgwnrmlxqzc>

La membresía del equipo de Supabase permite trabajar con el proyecto según el rol asignado. El acceso a la web se verifica por la tabla administradores y es independiente. Continuar en la sección 7.

6. Ruta B — Preparar un proyecto vacío

Este procedimiento se ejecuta una sola vez en una base nueva. Verificar el proyecto seleccionado antes de ejecutar SQL. Utilizar el SQL Editor de Supabase y pegar el contenido completo de cada archivo; ejecutar cada paso por separado y comprobar que termina correctamente.

Orden obligatorio:

1. supabase/migrations/202609230001_initial.sql
2. supabase/migrations/202609230002_private_admin_helper.sql
3. supabase/seed.sql
4. supabase/migrations/202609230003_demo_operations.sql

La migración 001 crea tablas, permisos, funciones y disparadores. La 002 mueve la comprobación administrativa al esquema privado. seed.sql incorpora las 300 lecturas originales. La 003 crea seis nodos virtuales, sus vínculos y tareas; falla si todavía no existen exactamente las 300 lecturas de la muestra.

No ejecutar las tres migraciones seguidas antes de cargar seed.sql. Este procedimiento usa SQL Editor y no registra automáticamente versiones en el historial de migraciones de la CLI. No combinarlo después con db push sin reconciliar ese historial. No volver a ejecutar CREATE TABLE sobre un esquema existente.

Alternativa a seed.sql, no un paso adicional: configurar temporalmente SUPABASE_SECRET_KEY en el entorno local y ejecutar npm run import:dataset después de la migración 002. El script usa data/smoke_detection_300.json, evita duplicados y verifica el total. La instalación normal usa los datos versionados; npm run dataset vuelve a descargar y muestrear la fuente y no es necesario para instalar.

Configurar autenticación y acceso inicial

En la configuración de Authentication, mantener habilitado el acceso por correo y contraseña y desactivar «Allow new users to sign up». Desactivar registro público no debe desactivar el proveedor de correo usado por las cuentas existentes. Referencia oficial: <https://supabase.com/docs/guides/auth/general-configuration>

Una base nueva requiere una identidad administrativa aprovisionada por el responsable; no puede iniciar sesión sin ella. Si existe una cuenta autorizada en Authentication, copiar su UUID y asignarle el rol desde SQL Editor:

```
insert into public.administradores (id, display_name)
values ('UUID_REAL_DEL_USUARIO', 'Administrador Satipo')
on conflict (id) do nothing;
```

Si no existe ninguna identidad, el responsable puede aprovisionar la primera fuera del registro público. El repositorio incluye scripts/bootstrap-admin.mjs para ese propósito; no ejecutarlo en la base compartida como parte de una instalación de compañero. Requiere ADMIN_EMAIL y la clave privada en el entorno, crea una cuenta y guarda la credencial inicial en config/admin.local.json. Ese archivo no debe compartirse ni subirse a GitHub. Una cuenta ya existente se autoriza con el SQL anterior, sin recrearla.

Para recuperación de contraseña, configurar las URL de retorno permitidas. La aplicación utiliza shared/account.html: permitir http://127.0.0.1:8000/shared/account.html para desarrollo y la URL equivalente del sitio desplegado. No cambiar la Site URL de producción solo para instalar un equipo local. Referencia: <https://supabase.com/docs/guides/auth/redirect-urls>

Comprobar la carga desde SQL Editor

```
select count(*) as total,  
       count(*) filter (where fire_alarm) as con_alarma,  
       count(*) filter (where not fire_alarm) as sin_alarma  
from public.smoke_readings  
where dataset_id = 'smoke-detection-iot-300-v1';
```

```
select node_id, count(*) as lecturas  
from public.demo_node_readings  
group by node_id order by node_id;
```

Resultado esperado: 300, 214 y 86 en la primera consulta; seis filas V-01 a V-06 con 50 lecturas cada una en la segunda. Las etiquetas positivas no equivalen a incendios confirmados. Continuar con la ejecución local.

7. Iniciar la aplicación

Desde la raíz del repositorio:

```
npm start
```

Mantener abierta la terminal. Abrir <http://127.0.0.1:8000/shared/login.html> e ingresar con la cuenta administrativa autorizada. No abrir [index.html](#) mediante `file://`, porque la aplicación carga configuración y recursos mediante HTTP.

Resultado esperado: ingreso a [/web/index.html](#), consulta de las 300 lecturas y navegación por nodos virtuales, incidentes y mantenimiento. Para detener el servidor, pulsar Ctrl+C. El servidor Python es de desarrollo, escucha solo en 127.0.0.1 y no sustituye un despliegue de producción.

Si el puerto está ocupado:

```
python scripts/serve.py --port 8001
```

Abrir <http://127.0.0.1:8001/shared/login.html>. Para recuperar contraseñas desde ese origen, autorizar también su URL de retorno en Supabase.

8. Validar la instalación

Comprobación manual sin modificar datos: iniciar sesión, verificar 300 lecturas, filtrar 214 con alarma y 86 sin alarma, abrir seis nodos virtuales y revisar las tarjetas de casos y mantenimiento. Comprobar desplazamiento vertical y cierre de sesión. No es necesario cambiar estados de la base compartida para acreditar la instalación.

Pruebas locales de base de datos:

```
npm test
```

Resultado esperado: dos pruebas aprobadas. Se ejecutan con PGlite en el equipo; no alteran la base remota ni certifican su configuración completa.

Pruebas de navegador opcionales: requieren Python con Playwright, Microsoft Edge y un archivo privado config/admin.local.json con email y password de una cuenta existente. Ese archivo no se descarga del repositorio. Para instalación de Playwright:

```
python -m pip install playwright  
python tests/panel_readings.py
```

El servidor local debe seguir activo. La prueba compara lecturas y verifica interfaz sin escribir datos. tests/panel_operations.py sí cambia temporalmente estados y los restaura; reservarlo para un entorno de pruebas o una ejecución coordinada. No ejecutar tests/e2e_cloud.py como comprobación rutinaria, porque incluye operaciones con cuentas y datos de prueba.

Guardar como evidencia propia las versiones instaladas, salida de npm test y pantalla de lecturas. No fotografiar contraseñas, tokens ni claves privadas. Las figuras de esta guía son explicativas, no capturas de ejecución.

9. Publicación opcional en Firebase

La ejecución local no necesita Firebase CLI. Para publicar, el operador debe tener acceso al proyecto correcto y firebase disponible en la terminal. Desde el repositorio configurado:

```
npm run build:hosting  
firebase login  
firebase deploy --only hosting --project alerta-satipo-76778920
```

El último comando reemplaza el sitio compartido; solo corresponde al responsable del despliegue. Para un entorno independiente, preparar su propio proyecto Firebase y usar su identificador explícito, no el anterior. `build:hosting` genera dist con configuración pública y excluye el componente móvil. No publicar la carpeta completa del repositorio.

Sitio existente: <https://alerta-satipo-76778920.web.app/>

10. Solución de problemas

- «Repository not found»: confirmar invitación al repositorio privado y cuenta usada por Git.
- npm o python no reconocido: revisar instalación y PATH; abrir una terminal nueva. Si solo existe py, instalar/configurar el alias python que utiliza npm start o ejecutar py scripts/serve.py.
- Opción --env-file-if-exists desconocida: actualizar Node 22 a una revisión reciente o una versión posterior compatible.
- «Falta configurar el aplicativo»: comprobar .env, ejecutar npm run configure y servir desde la raíz con npm start.
- Error de clave pública: usar publishable o anon del mismo proyecto que la URL; no usar secret/service_role.
- Credenciales inválidas: verificar la cuenta de Supabase Auth y contraseña; no confundirla con la contraseña de PostgreSQL o GitHub.
- Acceso denegado: comprobar que el UUID autenticado exista en administradores y tenga su perfil en clientes; revisar que se esté conectando al proyecto correcto.
- Relaciones demo inexistentes: en un proyecto nuevo, comprobar ejecución de la migración 003 después de cargar las 300 filas.
- Cero filas o error de red: comprobar sesión, proyecto activo, conexión y permisos RLS; no desactivar RLS para ocultar el error.
- Puerto 8000 ocupado: usar --port 8001 y la URL correspondiente.
- Recuperación envía a otro sitio: revisar la URL autorizada shared/account.html y configuración de correo.

11. Criterios de finalización y referencias

La instalación está completa cuando el servidor inicia, una cuenta autorizada accede, la muestra coincide con 300/214/86, existen seis lotes de 50 y npm test aprueba. La documentación de arquitectura hexagonal es un anexo de diseño; no es un requisito instalado ni una función del sistema actual.

Base técnica de esta guía: package.json, .env.example, scripts/configure.mjs, scripts/build.mjs, scripts/serve.py, scripts/import-dataset.mjs, scripts/bootstrap-admin.mjs y las tres migraciones SQL. Fecha de revisión: 25 de septiembre de 2026.

Supabase Auth: <https://supabase.com/docs/guides/auth/general-configuration>

URL de retorno: <https://supabase.com/docs/guides/auth/redirect-urls>

Los comandos y salidas descritas son instrucciones y resultados esperados. Las comprobaciones locales de generación del documento no equivalen a ejecutar desde cero la instalación en un equipo nuevo.